



Facultad de Ingeniería

Diseño de Aplicaciones 2

Obligatorio 1 – Evidencia de la aplicación de TDD y Clean Code

“Entregado como requisito para la obtención del título de Ingeniero en Sistemas”

Repo: <https://github.com/IngSoft-DA2/271395-294598-221509.git>

Candelaria Blanco - 271395

Maximiliano Gimnenez - 294598

Betina Kadessian - 221509

Tutores:

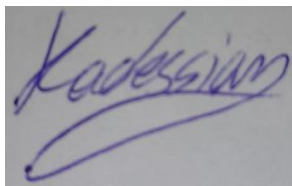
Nicolás Fierro, Maximo Healty

2025

DECLARACIÓN DE AUTORÍA

Nosotros, Candelaria Blanco, Maximiliano Gimenez y Betina Kadessian, declaramos que el trabajo que se presenta en esa obra es de nuestra propia mano. Podemos asegurar que:

- La obra fue producida en su totalidad mientras realizábamos el obligatorio 1 de la materia Diseño de Aplicaciones 2.
- Cuando hemos consultado el trabajo publicado por otros, lo hemos atribuido con claridad
- Cuando hemos citado obras de otros, hemos indicado las fuentes. Con excepción de estas citas, la obra es enteramente nuestra
- En la obra, hemos acusado recibo de las ayudas recibidas
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, hemos explicado claramente qué fue contribuido por otros, y qué fue contribuido por nosotros
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega, excepto donde se han realizado las aclaraciones correspondientes.



Ana Betina Kadessian

8/5/2025



Candelaria Blanco

8/5/2025



Maximiliano Gimenez

8/5/2025

ABSTRACT

Este trabajo presenta el desarrollo de un Simulador de Objetos que permite modelar y ejecutar estructuras propias de la Programación Orientada a Objetos (POO). El sistema posibilita la creación de clases con herencia simple, definición de atributos y métodos, y ejecución de métodos respetando accesibilidad, herencia y polimorfismo. También permite simular patrones de diseño como Template Method y Strategy. La solución fue implementada como una API REST utilizando .NET 8 y Entity Framework Core, siguiendo buenas prácticas de diseño, Clean Code y TDD. Este proyecto fue desarrollado para la Universidad ORT Uruguay como parte de la materia Diseño de Aplicaciones 2, con el objetivo de aplicar conceptos avanzados de diseño y arquitectura de software en un entorno práctico y educativo.

Índice

Estrategia de TDD seguida.....	4
Descripción.....	4
Cobertura de casos.....	6
Informe de cobertura para todas las pruebas desarrolladas	6
Obtenida utilizando Coverlet en GitHub Actions	6
Obtenida utilizando VS code.....	6
Obtenida utilizando JetBrains Rider	7
Análisis de las métricas de cobertura	7
Información sobre funcionalidades especificadas como prioritarias.....	8
Evidencia del ciclo de TDD	8
Requerimiento: Alta Baja Modificación de atributos (WebApi).....	8
Requerimiento: Alta Baja Modificación de atributos (Adapter)	8
Requerimiento: Alta Baja Modificación de atributos (Logic)	9
Requerimiento: Alta Baja Modificación de atributos (Data Access).....	9
Requerimiento: Alta de métodos que invoca un método (Webapi).....	10
Requerimiento: Alta de métodos que invoca un método (Adapter)	10
Requerimiento: Alta de métodos que invoca un método (Data Acces)	11
Requerimiento: Ejecutar método (Webapi).....	11
Requerimiento: Ejecutar método (Adapter)	11
Requerimiento: Ejecutar método (Logic)	12

CleanCode

Tratamos de seguir los estándares de Clean Code, aunque sabemos que nos faltan refactorizar muchas cosas, que debido a los límites de tiempo no tuvimos la oportunidad. Una de las cosas que nos gustaría refactorizar es la repetición de código en algunas funcionalidades. Subdividir en métodos más pequeños y reutilizarlo en varios lugares del código. También nos gustaría unificar un poco más el método de creación de una invocación del Adapter en un método de BussinessLogic y reducir más esa función para que sea más legible y mantenible.

Como aspectos positivos aplicamos:

Manejo de errores estructurado: Sistema de excepciones tipadas para diferentes escenarios (NonExistentValueLogic, InUseValueLogic)

Nombres descriptivos y significativos: Las clases, métodos y variables tienen nombres que claramente comunican su propósito

Métodos con responsabilidad única: Muchas de las operaciones están correctamente divididas según su función

Contratos claros entre capas: Las interfaces definen claramente qué esperar de cada componente

Consistencia en convenciones de código: Estilo uniforme en todo el proyecto

Organización lógica de namespaces: Estructura de carpetas y namespaces que refleja la arquitectura

Separación clara entre modelos de dominio y DTOs: Los objetos de transferencia de datos están separados de las entidades de dominio

Centralización de la lógica de validación: Validaciones importantes ubicadas en las capas de servicio

Estrategia de TDD seguida

Descripción

El proyecto adopta un enfoque integral de Test-Driven Development (TDD), estructurando las pruebas de manera organizada y siguiendo criterios consensuados en el equipo. A continuación, se presentan los principales lineamientos implementados:

- **Separación y organización de los tests**

La estructura de las pruebas refleja fielmente la del código fuente, distribuyéndose en carpetas específicas:

- *WebAPI*: Validación de controladores y *endpoints*.
- *Adapter*: Pruebas de la capa de adaptación entre la API y la lógica de negocio.
- *Domain*: Tests de entidades de dominio.
- *Business Logic*: Evaluación exhaustiva de reglas de negocio y validaciones.
- *Data Access*: Comprobación de persistencia y recuperación de datos.
- **Convención de nombres**
Las clases de prueba siguen el patrón [NombreClase]Test, facilitando la identificación entre las pruebas y sus respectivos componentes.
- **Herramientas utilizadas**
Se seleccionaron las siguientes tecnologías, consideradas apropiadas para los requerimientos del proyecto:
 - *MSTest*: Motor principal de ejecución de pruebas unitarias.
 - *Moq*: Biblioteca para la creación de objetos simulados (mocks), permitiendo la prueba de componentes de manera aislada.
 - *FluentAssertions*: Mejora la expresividad y legibilidad de las aserciones en los tests.
 - *Base de datos en memoria*: Solución empleada para pruebas de acceso a datos sin dependencia de bases externas.
- **Metodología de pruebas por clase**
Cada clase de prueba sigue una estructura metodológica común:
 - *Configuración inicial*: Se emplean métodos [TestInitialize] para establecer el entorno de prueba, incluyendo la preparación de mocks y objetos necesarios.
 - *Pruebas específicas*: Cada método de prueba aborda un único comportamiento o caso de uso, asegurando la claridad y la precisión.
- **Estrategia utilizada:**
El enfoque seguido fue mayormente *outside-in*, comenzando por definir las pruebas en la capa *WebApi* e ir refinando las capas internas a medida que los test lo requerían, aunque en algunos casos específicos se aplicó *inside-out* para definir primero el comportamiento de entidades o servicios para facilitar el trabajo.

Cobertura de casos

Para la cobertura de casos utilizamos lo siguiente, implementándolos cuando nos parecían pertinentes y alineados con las necesidades del proyecto:

- *Casos de éxito*: Se verifican los comportamientos esperados en situaciones normales.
- *Casos de error*: Se emplea extensivamente *Assert.ThrowsException* para comprobar el correcto manejo de excepciones.
- *Casos límite*: Pruebas para situaciones particulares o escenarios complejos. Informe de cobertura para todas las pruebas desarrolladas

Obtenida utilizando Coverlet en GitHub Actions

Assembly	Line Coverage	Branch Coverage	Health (over 90.0%)
Adapter	98.47%	86.96%	✗
BussinesLogic	96.67%	94.55%	✓
DataAccess	97.64%	91.05%	✓
Domain	98.39%	93.55%	✓
Models	100.00%	50.00%	✗
WebApi	100.00%	100.00%	✓
Summary	**98.53%**	**86.02%**	✗

Minimum allowed code coverage is 90.0%

✓ Summary line coverage meets the threshold (98.53% ≥ 90.0%).

Obtenida utilizando VS code

src	42%	DataAccess	24%
> Adapter	97%	> bin	
> BussinesLogic	96%	> Context	100%
> DataAccess	24%	> Migrations	0%
> Domain	98%	> obj	
> IAdapter		⚠ DataAccess.csproj	
> IBussinesLogic		🔗 ExecutionDataAccess.cs	95%
> IDataAccess		🔗 SimAttributeDataAccess.cs	100%
> Models	100%	🔗 SimClassDataAccess.cs	99%
> ServiceFactory		🔗 SimMethodDataAccess.cs	98%
> WebApi	82%	📁 WebApi	82%
📄 .dockerignore		> bin	
📄 docker-compose.yml		> Controllers	100%
> TestResults		> Filters	100%
> tests	98%	> obj	
		> Properties	
		📄 appsettings.Development.json	
		📄 appsettings.json	
		🔗 Program.cs	0%

Obtenida utilizando JetBrains Rider

Symbol	Coverage (%)	Uncovered/Total Stmts.
✓ Total	83%	1247/7242
> tests	97%	122/4185
✓ src	63%	1125/3057
> Models	96%	4/101
> Adapter	96%	15/393
> BussinesLogic	96%	12/276
> Domain	95%	23/492
✓ WebApi	76%	35/147
> {} WebApi	100%	0/112
> Program	0%	35/35
✓ DataAccess	37%	1036/1648
✓ {} DataAccess	37%	1036/1648
> SimAttributeDataAccess	100%	0/32
> {} Context	100%	0/97
> SimClassDataAccess	99%	2/150
> SimMethodDataAccess	97%	4/143
> ExecutionDataAccess	95%	10/206
> {} Migrations	0%	1020/1020

Análisis de las métricas de cobertura

Se identifican dos componentes con cobertura nula (0 %) que requieren análisis contextual: el archivo *Program.cs* y la carpeta *Migrations*.

El archivo *Program.cs*, dentro del módulo *WebApi*, tiene una cobertura del 0 % con 35 líneas no cubiertas. Esta situación es esperable, dado que dicho archivo se encarga de la inicialización y configuración de la aplicación (registro de servicios, configuración de middleware, entre otros). Estas tareas pertenecen al dominio de pruebas de integración o pruebas end-to-end, y no suelen ser cubiertas por pruebas unitarias. Por tanto, su exclusión del análisis no compromete la validez de las métricas de calidad del código funcional.

Por otro lado, el submódulo *Migrations* dentro de *DataAccess* también muestra una cobertura del 0 %, con un total de 1020 líneas sin cubrir. Estas líneas corresponden a archivos generados automáticamente por el sistema de migraciones de Entity Framework. Este tipo de código no contiene lógica personalizada y, en consecuencia, no se justifica su inclusión dentro del alcance de las pruebas unitarias.

A pesar de estos casos particulares, los módulos funcionales principales (*BusinessLogic*, *Adapter*, *Domain*, e incluso partes relevantes de *DataAccess*) presentan niveles de cobertura superiores al 95 %, lo que garantiza que la lógica de negocio y la capa de persistencia están debidamente validadas mediante pruebas.

En resumen, el informe de cobertura es sólido: las áreas con baja cobertura se explican por su naturaleza técnica o por ser código autogenerado, mientras que los componentes críticos del sistema tienen una cobertura ampliamente satisfactoria.

Información sobre funcionalidades especificadas como prioritarias

Evidencia del ciclo de TDD

Requerimiento: Alta Baja Modificación de atributos (WebApi)

(Pull Request Feature/attribute class #5)

RED: CreateAttributeWithEmptyOrWhiteSpace_ShouldThrowSimClassInvalidAttribute	8743779	<>
GREEN: CreateAttributeWithEmptyOrWhiteSpace_ShouldThrowSimClassInvalidAttribute	88b3a38	<>
RED: CreateAttributeWithInvalidCharacters_ShouldThrowSimClassInvalidAttribute	6fc3273	<>
GREEN: CreateAttributeWithInvalidCharacters_ShouldThrowSimClassInvalidAttribute	6cc2e39	<>
RED: CreateAttributeWithOnlyNumbers_ShouldThrowSimClassInvalidAttribute	56f37d6	<>
GREEN: CreateAttributeWithOnlyNumbers_ShouldThrowSimClassInvalidAttribute	d8b31fb	<>
RED: CreateAttributeWithReservedWords_ShouldThrowSimClassInvalidAttribute	4782649	<>
GREEN: CreateAttributeWithReservedWords_ShouldThrowSimClassInvalidAttribute	6c388f9	<>
RED: AddCorrectAttributeToSimClass_CorrectlyAddsIt	6a332ca	<>
GREEN: AddCorrectAttributeToSimClass_CorrectlyAddsIt	87f329c	<>
RED: AddAttributeWithRepetedNameClass_ShouldThrowSimClassInvalidAttribute	d984a41	<>
GREEN: AddAttributeWithRepetedNameClass_ShouldThrowSimClassInvalidAttribute	c57b4d2	<>
RED: DeleteExistentAttribute_ShouldDeleteIt	533862d	<>
GREEN: DeleteExistentAttribute_ShouldDeleteIt	e6cc5f9	<>
RED: DeleteInexistentAttribute_ShouldThrowException	27db733	<>
GREEN: DeleteInexistentAttribute_ShouldThrowException	2cc2e5d	<>
RED: DeleteAttributeControllerWithCorrectId_ShouldDeleteOk	77b289b	<>
GREEN: DeleteAttributeControllerWithCorrectId_ShouldDeleteOk	d3a28d8	<>
GREEN: UpdateAttributeControllerWithCorrectBody_ShouldUpdateOk	fabc33a	<>
REFACTOR: UpdateAttributeControllerWithCorrectBody_ShouldUpdateOk	859fed8	<>

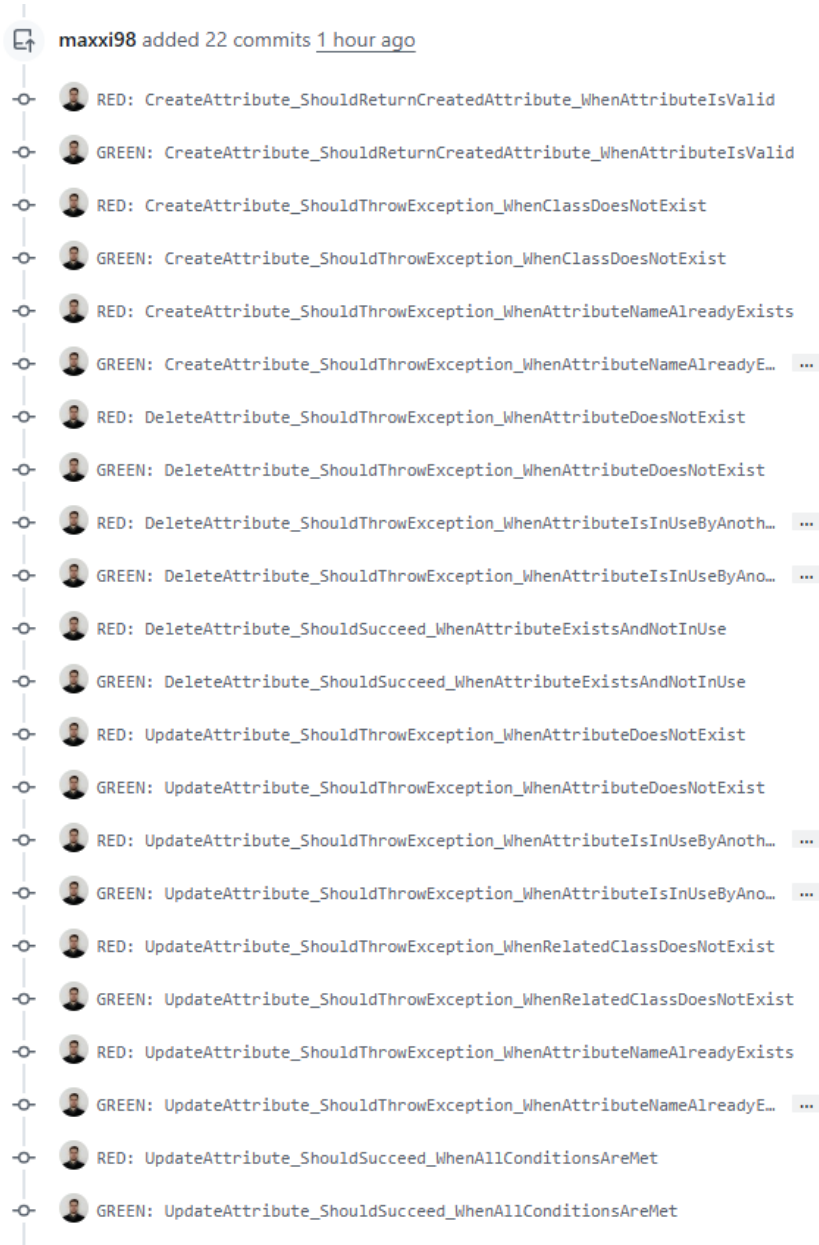
Requerimiento: Alta Baja Modificación de atributos (Adapter)

(Pull request #22)

RED: DeleteAttribute_CallsServiceDeleteAttribute_WithCorrectId	committed 1 hour ago
GREEN: DeleteAttribute_CallsServiceDeleteAttribute_WithCorrectId	committed 1 hour ago
RED: DeleteAttribute_WhenServiceThrowsException_ThrowsInvalidOperationException	committed 1 hour ago
GREEN: DeleteAttribute_WhenServiceThrowsException_ThrowsInvalidOperationException	committed 1 hour ago
RED: UpdateAttribute_WithValidData_UpdatesAndReturnsResponse	committed 1 hour ago
GREEN: UpdateAttribute_WithValidData_UpdatesAndReturnsResponse	committed 1 hour ago
RED: UpdateAttribute_WhenServiceThrowsException_ThrowsObjectNotFoundException	committed 1 hour ago
GREEN: UpdateAttribute_WhenServiceThrowsException_ThrowsObjectNotFoundException	committed 1 hour ago
RED: CreateAttribute_WithValidData_CreatesAndReturnsResponse	committed 1 hour ago
GREEN: CreateAttribute_WithValidData_CreatesAndReturnsResponse	committed 1 hour ago
RED: CreateAttribute_WhenServiceThrowsException_ThrowsObjectNotFoundException	committed 52 minutes ago
GREEN: CreateAttribute_WhenServiceThrowsException_ThrowsObjectNotFoundException	committed 52 minutes ago

Requerimiento: Alta Baja Modificación de atributos (Logic)

(Pull Request feature/attribute logic #25)



Requerimiento: Alta Baja Modificación de atributos (Data Access)

(Pull request #38 from IngSoft-DA2/feature/attributes-dataAccess)

↻ [red] create attribute test	31fc32b
↻ [green] create attribute test	181728d
↻ [refactor] create attribute test bd problems case	e261177
↻ [red] delete attribute test	6428eae
↻ [green] delete attribute test	9d43a6d
↻ [refactor] delete attribute test database problem case	69d52b2
↻ [red] existattribute by id test	ae610f
↻ [green] existattribute by id test	d95106b
↻ [refactor] existattribute by id test db problem case	47c9999
↻ [red] exist attribute by name test	2e68fa1
↻ [green] exist attribute by name test	b4fc2cb
↻ [refactor] exist attribute by name test db problem case	fdca9a7
↻ [red] update attribute test	58baf41
↻ [green] update attribute test	310a0e0
↻ [refactor] update attribute test db problem case	c2770e0
↻ [red] isUsedByOther at an invocation test	be0f02e
↻ [green] isUsedByOther at an invocation test	01de2a2
↻ [refactor] isUsedByOther at an invocation db problem case	73c36ed
↻ fix: improvements for higher coverage	ac95cea
↻ files format for pull request passing	f0065f8

Requerimiento: Alta de métodos que invoca un método (Webapi)
(Pull request Feature/Controller method invocations #15)

Merge pull request #15 from IngSoft-DA2/feature/methodExecutions
betinakd authored 2 weeks ago · ✓ 6 / 6
Merge branch 'develop' into feature/methodExecutions
betinakd authored 2 weeks ago · ✓ 6 / 6
GREEN: CreateMethodInvocationCorrectly_ShoulReturnCreated
betinakd committed 2 weeks ago · ✓ 6 / 6
RED: CreateMethodInvocationCorrectly_ShoulReturnCreated}
betinakd committed 2 weeks ago
GREEN: GetInvocationMethodCorrectly_ShouldThrowOk
betinakd committed 2 weeks ago
RED: GetInvocationMethodCorrectly_ShouldThrowOk
betinakd committed 2 weeks ago

Requerimiento: Alta de métodos que invoca un método (Adapter)
(Pull request Feature/Controller method invocations #21)

Merge pull request #21 from IngSoft-DA2/feature/AdapterManagerInvocation 

 betinakd authored 2 weeks ago ·  6 / 6

GREEN: GetWrongInvocation_ShouldThrowInvalidOperationException

 betinakd committed 2 weeks ago ·  6 / 6

RED: GetWrongInvocation_ShouldThrowInvalidOperationException

 betinakd committed 2 weeks ago

GREEN: GetInvocation_ShouldReturnInvocationResponse_WhenValid

 betinakd committed 2 weeks ago

RED: GetInvocation_ShouldReturnInvocationResponse_WhenValid

 betinakd committed 2 weeks ago

GREEN: CreateWrongInvocation_ShouldThrowInvalidOperationException

 betinakd committed 2 weeks ago

RED: CreateWrongInvocation_ShouldThrowInvalidOperationException

 betinakd committed 2 weeks ago

GREEN: CreateInvocation_ShouldReturnCreatedInvocationResponse_WhenValid

 betinakd committed 2 weeks ago

RED: CreateInvocation_ShouldReturnCreatedInvocationResponse_WhenValid

 betinakd committed 2 weeks ago

Requerimiento: Alta de métodos que invoca un método (Data Acces)
(Pull request #28)

  **RED: GetInvocationById_ShouldThrowException_WhenInvocationDoesNotExist**

  **GREEN: GetInvocationById_ShouldThrowException_WhenInvocationDoesNotExist**

  **RED: GetInvocationById_ShouldReturnInvocation_WhenInvocationExists**

  **GREEN: GetInvocationById_ShouldReturnInvocation_WhenInvocationExists**

Requerimiento: Ejecutar método (Webapi)
(Pull request #17 ExecuteMethod Controller)

RED: ExecuteMethodOk_ReturnsObjectWithExecution

 betinakd committed 10 minutes ago

GREEN: ExecuteMethodOk_ReturnsObjectWithExecution

 betinakd committed 10 minutes ago ·  6 / 6

Requerimiento: Ejecutar método (Adapter)
(Pull request #23 ExecuteMethod Adapter)

RED: ExecuteMethodValidInputs_ShouldMapParametersAndCallExecutionService
 betinakd committed 18 minutes ago
GREEN: ExecuteMethodValidInputs_ShouldMapParametersAndCallExecutionSe...
 betinakd committed 17 minutes ago
RED: ExecuteMethod_WhenExceptionThrown_ShouldThrowInvalidExecutionExc...
 betinakd committed 6 minutes ago
GREEN: ExecuteMethod_WhenExceptionThrown_ShouldThrowInvalidExecutionE...
 betinakd committed 6 minutes ago

Requerimiento: Ejecutar método (Logic)

(Pull request #41 from IngSoft-DA2/feature/ExecuteMethod)

GREEN: ExecuteMethod_RecursiveCall_DetectsRecursion

 betinakd committed 19 hours ago

RED: ExecuteMethod_RecursiveCall_DetectsRecursion

 betinakd committed 19 hours ago

GREEN: ExecuteMethod_NestedInvocations_FormatsProperly

 betinakd committed 19 hours ago

RED: ExecuteMethod_NestedInvocations_FormatsProperly

 betinakd committed 19 hours ago

GREEN: ExecuteMethod_MethodNotFound_ReturnsErrorMessage

 betinakd committed 19 hours ago

RED: ExecuteMethod_MethodNotFound_ReturnsErrorMessage

 betinakd committed 19 hours ago

GREEN: ExecuteMethod_BasicMethodWithoutInvocations_ReturnsFormattedOutput

 betinakd committed 19 hours ago

RED: ExecuteMethod_BasicMethodWithoutInvocations_ReturnsFormattedOutput

 betinakd committed 19 hours ago

GREEN: TestFindMethodInHierarchy_PrivateMethodsNotInherited

 betinakd committed yesterday

RED: TestFindMethodInHierarchy_PrivateMethodsNotInherited

 betinakd committed yesterday

GREEN: TestFindMethodInHierarchy_ThroughPublicMethods

 betinakd committed yesterday

RED: TestFindMethodInHierarchy_ThroughPublicMethods

 betinakd committed yesterday

GREEN: MatchSignature method

 betinakd committed yesterday